

T.C.

[ÜNİVERSİTE ADI]

[FAKÜLTE ADI]

STAJ DEFTERİ

OpenNutri

Sesli Besin Kaydı ve Beslenme Takibi
Mobil Uygulaması

Bölümü	[BÖLÜM ADI]
Öğrenci Numarası	[ÖĞRENCİ NUMARASI]
Adı Soyadı	[AD SOYAD]
Çalışılan Kurum	[KURUM ADI]
Çalışma Devresi	Yaz Stajı
Tarih Aralığı	[GG/AA/YYYY] – [GG/AA/YYYY]

30 günlük içerik taslağı • Tarih ve katkı doğrulaması bekliyor

Kurumun resmî kapak ve onay formu varsa bu kapağın yerine kullanılmalıdır.

PRATİK ÇALIŞMA BİLGİLERİ

Öğrenci Adı Soyadı	[AD SOYAD]
Öğrenci Numarası / Bölümü	[ÖĞRENCİ NUMARASI] / [BÖLÜM ADI]
Kurum Adı	[KURUM ADI]
Kurum Adresi	[KURUM ADRESİ]
Sorumlu Adı Soyadı	[SORUMLU AD SOYAD]
Sorumlu Unvanı	[SORUMLU UNVANI]
Başlangıç / Bitiş Tarihi	[GG/AA/YYYY] / [GG/AA/YYYY]
Günlük İçerik Sayısı	30 — fiilî çalışma günleriyle eşleştirilecek

Belgenin kapsamı

Bu defter yalnızca OpenNutri mobil uygulamasını anlatır. Mevcut besin ve yapay zekâ servisleri, uygulamanın kullandığı dış bileşenler olarak ele alınmıştır. Veri kaynağının oluşturulması ve diğer proje çalışmalarına ilişkin geliştirme iddiası içermez.

Teslim öncesi doğrulama

Birinci tekil şahısla yazılan günlükler, öğrenci tarafından doğrulanacak anlatım önerileridir. Yapılan işi açıklayan kod ve testler, o işi öğrencinin yaptığını veya belirtilen sırada çalıştığını tek başına kanıtlamaz. Öğrenci, geliştirme, entegrasyon, test ve inceleme ifadelerini gerçek katkısına göre düzeltmeli; tarihleri doğrulamalıdır. Sayısal örnekler kodu açıklayan kontrollü verilerdir. Metin yapay zekâ desteğiyle hazırlanmış; kurum bilgileri ve imzalar boş bırakılmıştır.

Onay alanı — ilgili yetkili tarafından doldurulur

Adı Soyadı / Unvanı:

Tarih: İmza ve Mühür:

Uygulama Konusunun ve Kullanıcı İhtiyaçlarının Belirlenmesi

Bugün OpenNutri mobil uygulamasının çözmek istediği problemi ve temel kullanım akışını ele aldım. Beslenme günlüğü tutan bir kişinin aynı öğündeki besinleri ayrı ayrı araması, miktar girmesi ve her adımı onaylaması gerektiğinde kayıt işlemi uzuyor. Bu nedenle çalışmanın merkezine, besini hızlı kaydetme ve yanlış kaydı sonradan kolayca düzeltme ihtiyacını koydum. Uygulamanın yalnızca kalori gösteren bir ekran değil, günlük kullanımı sürdürülebilir kılan bir araç olmasını hedefledim.

İşlevleri besin arama, porsiyon belirleme, öğün günlüğü, günlük toplamlar ve kişisel hedefler şeklinde ayırdım. Sesli kayıt ve ana ekran mikrofonu, aynı günlük kayıt sistemine ulaşan kısa yollar olarak planlandı. Koç ve Oracle ise kaydedilmiş bilgilerden yararlanan öneri ekranlarıdır. Böylece her özellik için ayrı bir besin listesi veya farklı bir hesaplama yöntemi oluşturmak yerine, ortak veri modelini kullanacak bir yapı ortaya çıktı.

Örnek kullanıcı akışını “besini bul, miktarı gir, kaydet, toplamı gör, gerekirse düzelt” şeklinde kurdum. Başarı ölçütünü yalnızca kaydın ekranda görünmesiyle sınırlamadım; uygulama yeniden açıldığında korunması, yanlış güne yazılmaması ve düzenleme iptal edildiğinde kaybolmaması da gerekiyordu. Sesli kayıta miktar tahmini yapılırsa bunun kullanıcıya gösterilmesini, eşleşmeyen bir besinin ise sessizce doğru kabul edilmemesini benimsedim.

Bu çalışma sırasında yapay zekânın göreviyle besin verisinin kaynağını özellikle ayırdım. Yapay zekâ ifadeyi anlamaya yardımcı olurken sayısal değerler mevcut Core servisinden alınmaktadır. Gün sonunda uygulamanın kapsamını Android öncelikli mobil geliştirme olarak netleştirdim; veri seti üretimi ve araştırma sistemlerini bu çalışmaya dahil etmedim. Kullanıcı ihtiyacını somut kabul koşullarına dönüştürmenin, sonraki geliştirmeleri değerlendirmeyi kolaylaştırdığını öğrendim.

Çalışılan Tarih	Onaylayanın Adı Soyadı	İmza ve Mühür
[GG/AA/YYYY]	[SORUMLU AD SOYAD]	

Flutter Geliştirme Ortamı ve Dart Proje Yapısı

Bugün projenin geliştirme araçlarını ve dosya düzenini çalıştım. Flutter'ın arayüzü bileşenlerden oluşturduğunu, Dart'ın ise bu bileşenlerin davranışını ve veri işlemlerini tanımladığını öğrendim. Android SDK ve Gradle, yazılan kodun telefona kurulabilir pakete dönüştürülmesinde kullanılmaktadır. Bu ayrımı yapmak, bir hata çıktığında sorunun Dart kodunda mı, kullanılan pakette mi yoksa Android derleme ayarlarında mı olduğunu araştırmamı kolaylaştırdı.

Öncelikle pubspec.yaml dosyasındaki bağımlılıkları görevlerine göre ayırdım. http ağ istekleri, shared_preferences yerel saklama, record mikrofon kaydı, path_provider ise geçici dosya dizini için kullanılıyor. Kimlik doğrulama için Supabase bağlantısı bulunuyor. Bir paketi projeye eklemenin tek başına özelliği tamamlamadığını gördüm; paketin hata durumlarını yakalamak, sonuçlarını uygulamanın modellerine çevirmek ve ekran kapanırken kaynaklarını bırakmak da gerekiyor.

Komutların hangi aşamayı doğruladığını not ettim. flutter pub get bağımlılıkları hazırlar; flutter analyze kaynak kodu statik olarak denetler; flutter test beklenen davranışları sınar. flutter build apk --release ise Android kurulum paketini üretir. Analizin temiz çıkmasıyla telefondaki bütün etkileşimlerin doğrulanmasının aynı şey olmadığını özellikle ayırdım. Derleme çıktısını kaynak kod yerine düzenlememek ve kalıcı değişiklikleri lib ile android altındaki ilgili dosyalarda yapmak gerektiğini öğrendim.

Son olarak yapılandırma değerlerinin --dart-define üzerinden aktarılmasını inceledim. Uygulamadaki herkese açık istemci anahtarı ile sunucuda tutulması gereken Gemini veya yönetici anahtarları aynı yetkiye sahip değildir. Gizli anahtarları kaynak dosyalara yazmadan çalışmak, Git değişikliklerini kontrol etmek ve doğrulanmış sürümleri korumak geliştirme düzeninin bir parçası oldu. Gün sonunda araçların görevlerini ve projenin hangi bölümüne ne zaman müdahale edeceğimi daha iyi anladım.

Çalışılan Tarih	Onaylayanın Adı Soyadı	İmza ve Mühür
[GG/AA/YYYY]	[SORUMLU AD SOYAD]	

Uygulamanın Katmanlara Ayrılması

Bugün ekran kodu ile veri işlemlerinin birbirine karışmasını önleyen uygulama yapısı üzerinde çalıştım. Besin adı, miktarı ve günlük kayıt bilgileri model sınıflarında; ağ bağlantısı ve yerel saklama servislerde tutuluyor. AppController ise seçilen gün, öğünler, hedefler ve kayıt değişiklikleri gibi ortak durumu yönetiyor. Bu düzen sayesinde bir düğmenin görünümünü değiştirirken kayıt biçimini de değiştirmek zorunda kalmıyorum.

Bir besin ekleme işlemini adım adım takip ettim. Ekran bir DiaryEntry üretip denetleyiciye iletiyor; denetleyici listeyi güncelliyor, notifyListeners ile dinleyen ekranları haberdar ediyor ve LocalStore üzerinden saklıyor. Günlük toplamalarını gösteren ekranın ayrıca bağımsız bir kayıt listesi tutmaması önemliydi. Aynı verinin iki yerde ayrı ayrı değiştirilmesi, bir ekranda güncel diğerinde eski toplam görünmesine neden olabilirdi.

Başlangıçta main.dart içinde Flutter bağlamının hazırlanması, yerel kayıtların yüklenmesi ve uygulamanın açılması sırasını inceledim. Supabase hazırlığı beklenmeden başlatılıyor; günlük arayüzü uzak kimlik doğrulama servisini beklemiyor. Yerel denetleyicinin hazırlanması ise runApp öncesinde tamamlanıyor. Böylece daha önce kaydedilmiş öğünlerin gösterilmesiyle yapay zekâ bağlantısının hazır olması birbirinden ayrılmış oluyor.

Testlerde gerçek depolama veya ağ istemcisi yerine kontrollü nesneler verilebilmesini de kullandım. Örneğin LocalStore yerine kaydetmeyi bekleten bir test nesnesi, normal kullanımda yakalanması zor yazma sırası sorunlarını görünür kılıyor. Gün sonunda katmanlı yapının yalnızca dosyaları klasörlere ayırmak olmadığını; her bileşenin girdisini, çıktısını ve sorumluluğunu belirlemek anlamına geldiğini öğrendim.

Tablo 1. Mobil uygulamadaki sorumluluk ayrımı

Modeller	Besin, günlük kaydı, hedef ve profil bilgileri
Denetleyici / ekranlar	Ortak durum ve kullanıcı etkileşimleri
Servisler	Yerel saklama ve mevcut HTTP servislerine erişim

Çalışılan Tarih	Onaylayanın Adı Soyadı	İmza ve Mühür
[GG/AA/YYYY]	[SORUMLU AD SOYAD]	

Ana Ekran, Tema ve Sayfalar Arası Geçiş

Bugün uygulamanın ana ekranını ve sayfalar arasındaki geçişleri düzenledim. HomeShell içerisinde günlük, besin öğeleri, koç, Oracle ve ayarlar olmak üzere beş ana bölüm bulunuyor. Kullanıcının en sık ihtiyaç duyduğu besin arama ve sesle kayıt işlemlerini günlük ekranından erişilebilir tuttum. Diyet seçimini ayrı bir ekran olarak bağlayarak ana gezinme çubuğuna gereğinden fazla seçenek eklememeye dikkat ettim.

Arayüzde ortak tema, kartlar, düğmeler ve öğün bölümlerini kullandım. Günlük özet ve öğün listelerinin aynı tarih üzerinden hesaplanması için AppController'ın seçilen gün bilgisini esas aldım. Tarih ileri veya geri alındığında başlıkla birlikte kayıtların ve toplamların da değişmesi gerekiyor. Yalnızca tarih metnini değiştirmek, kullanıcıya başka güne ait beslenme bilgisi gösterilmesine yol açacağından yeterli değildir.

Boş bir gün, kayıt bulunan bir gün ve veri beklenen ekranlar için farklı görünüm durumlarını ele aldım. Boş ekranda kullanıcının ilk kaydı nasıl yapacağını anlaması; kayıt bulunan ekranda ise toplamlarla öğünlere kolay ulaşması amaçlandı. Kaydırılabilir alanlar ve içerik genişliği sınırı, uzun besin adlarının ve farklı ekran ölçülerinin düzeni bozmasını azaltıyor. Renk ve simgelerin yanında açıklayıcı metin kullanarak işlemlerin anlaşılır kalmasını sağladım.

Ana ekranın kayıt davranışını home_shell_test.dart içindeki senaryoyla değerlendirdim. Depolama tamamlanmadan yeni besinin görünmesi, arayüzün gereksiz beklemediğini gösteriyor; kalıcı kaydın sonucu ise denetleyicide ayrıca takip ediliyor. Bu çalışma sayesinde görsel tasarımla veri akışını birlikte düşünmeyi öğrendim. Güzel görünen bir ekranın, yanlış gün veya eski veri göstermesi durumunda işlevsel açıdan doğru sayılamayacağını gördüm.

Çalışılan Tarih	Onaylayanın Adı Soyadı	İmza ve Mühür
[GG/AA/YYYY]	[SORUMLU AD SOYAD]	

Besin ve Günlük Kayıt Modellerinin Oluşturulması

Bugün besin arama sonuçları ile günlüğe kaydedilen öğünlerin veri modellerini çalıştım. Arama listesinde yalnızca seçim için gereken kısa bilgiler bulunurken FoodDetail sınıfı porsiyonları, kaynak bilgilerini ve besin öğelerini taşıyor. DiaryEntry ise kullanıcının belirli bir gün ve öğünde tükettiği miktarı temsil ediyor. Aynı besin farklı günlerde yenebildiğinden, kaynak besinin kimliğiyle günlük kaydın kimliğini ayrı tuttum.

Kayıtta foodId, foodName, dateKey, meal, grams ve nutrients gibi alanları kullandım. inputGrams kullanıcının girdiği ağırlığı, grams hesaplamaya esas yenilebilir ağırlığı tutuyor. weightBasis bu iki değerden neden farklı olabileceğini açıklıyor. loggedByVoice kaydın sesle oluştuğunu, needsReview ise sonradan gözden geçirilmesi gerektiğini belirtiyor. Bu işaretler sayesinde tahmini bir kayıt, normal kayıtlarla aynı günde bulunurken belirsizliği görünür kalıyor.

DiaryEntry.fromFood içinde yalnızca yüz gram yenilebilir kısım temeline sahip besin değerlerini miktara göre ölçekledim. Yoğunluk gibi fiziksel özellikler aynı şekilde tüketilen besin miktarı olarak toplanmamalıdır. Kayda hesaplanmış değerlerin anlık kopyasını eklemek de önemliydi: eski öğünün değerlerini her açılışta uzak servisten yeniden üretmek, kaynak değiştiğinde günlüğün geçmişini değiştirebilirdi.

toJson ve fromJson dönüşümlerinde tarih, kimlik, miktar ve besin listesinin korunmasını kontrol ettim. Eski kayıtlarda inputGrams yoksa grams değerinin kullanılması, yeni alanların varsayılanlarla okunabilmesine örnektir. JSON dönüşüm testinde 125 gramlık kaydın geri okunduğunda aynı kimliği ve kaloriyi taşıması bekleniyor. Gün sonunda veri modelini tasarlarken yalnızca bugünkü ekranı değil, sonraki sürümlerde eski kayıtların okunmasını da düşünmek gerektiğini öğrendim.

Çalışılan Tarih	Onaylayanın Adı Soyadı	İmza ve Mühür
[GG/AA/YYYY]	[SORUMLU AD SOYAD]	

Besin Arama Servisinin Uygulamaya Bağlanması

Bugün mobil uygulamanın mevcut besin servisiyle iletişimini sağlayan CoreApiClient üzerinde çalıştım. Arama için GET /v1/foods/search, seçilen besinin ayrıntıları için GET /v1/foods/{foodId} uç noktaları kullanılıyor. Uygulamayı doğrudan veritabanı tablolarına bağlamak yerine bu HTTP sözleşmesini esas aldım. Böylece verinin sunucuda nasıl saklandığını bilmeden, uygulamanın hangi isteği göndereceği ve hangi alanları okuyacağı açık hâle geldi.

searchFoods içinde arama metninin başındaki ve sonundaki boşlukları temizledim. Metin boşsa ağ isteği oluşturmadan boş sonuç dönülüyor. Dolu sorguda q ve limit parametreleri URI üzerinden hazırlanıyor; varsayılan sonuç sınırı 30. Yanıtın JSON içeriği FoodSearchResults modeline çevriliyor. Yanıt kodu başarılı aralıkta değilse hata üretilmesi, bağlantı sorununun yanlışlıkla “besin bulunamadı” diye yorumlanmasını engelliyor.

Arama sonuçlarından seçim yapıldığında önce besinin ayrıntısını yükleyip ardından porsiyon penceresini açan akışı bağladım. Kullanıcı porsiyonu onaylarsa bu pencere DiaryEntry döndürüyor; iptal ederse kayıt oluşmuyor. Ayrıntıları besin kimliğine göre bellekte saklayan önbellek, aynı besin tekrar açıldığında gereksiz ağ çağrılarını azaltıyor. Bu önbelleğin uygulama kapanınca bütün besinleri koruyan çevrimdışı bir veritabanı olmadığını ayırdım.

İsteklere 30 saniyelik zaman aşımı uygulanmasını ve hatanın arayüzde yakalanmasını inceledim. Beklenen akışı “sorgu → kısa sonuç → ayrıntı → porsiyon → günlük kayıt” şeklinde takip ederek veri dönüşümlerini netleştirdim. Bu çalışma, API entegrasyonunun yalnızca adres çağırmak olmadığını gösterdi; boş girdi, başarısız yanıt ve kullanıcının işlemi iptal etmesi de akışın parçalarıdır. Besin servisinin geliştirilmesini değil, hazır servisin mobil uygulamaya bağlanmasını üstlenen katman üzerinde çalıştım.

Çalışılan Tarih	Onaylayanın Adı Soyadı	İmza ve Mühür
[GG/AA/YYYY]	[SORUMLU AD SOYAD]	

Asenkron Arama ve Eski Sonuçların Engellenmesi

Bugün arama ekranında hızlı yazı yazıldığında ortaya çıkabilen eski sonuç sorununu ele aldım. Kullanıcı "apple" sorgusundan hemen sonra "chicken" yazarsa ilk isteğin daha geç tamamlanması mümkündür. Yanıtları geliş sırasına göre doğrudan ekrana koymak, arama kutusunda chicken yazarken elma sonuçlarının görünmesine neden olabilir. Bu sorunun yalnızca görünümle ilgili olmadığını, yanlış besinin seçilmesine de yol açabileceğini gördüm.

Öncelikle her tuş vuruşunda istek göndermemek için 350 milisaniyelik gecikmeli aramayı kullandım. Yeni bir değişiklik olduğunda önceki zamanlayıcı iptal ediliyor ve yeni süre başlıyor. Ancak bu yöntem, daha önce gönderilmiş bir isteği tek başına geçersiz kılmıyor. Bu nedenle her değişiklikte istek numarasını artırıp, dönen yanıtın numarasını güncel numarayla karşılaştırdım. Yalnızca son aramaya ait yanıt ekran durumunu güncelliyor.

Arama alanı temizlendiğinde de numaranın artırılması gerekiyordu. Aksi hâlde liste boşaltıldıktan sonra yoldaki eski yanıt gelip sonuçları tekrar doldurabilirdi. Normal aramayla gönderilmiş metnin yapay zekâ çözümlemesini ayrı sayaçlarla takip ettim. Ekran kapanmışsa güncelleme yapılmaması için mounted kontrolünü ekledim; zamanlayıcı ve metin dinleyicileri de dispose sırasında bırakılıyor.

Regresyon testinde isteğin yanıtını Completer ile bekletip arama alanını temizleyen senaryoyu kullandım. Sonradan tamamlanan yanıtın ekranda besin göstermemesi bekleniyor. Böylece yalnızca hızlı internet üzerinde düzgün görünen bir ekranı değil, yanıt sırası değiştiğinde de doğru davranan akışı değerlendirdim. Gün sonunda asenkron programlamada en son tamamlanan işlemin her zaman kullanıcının en son isteği olmadığını öğrendim.

Çalışılan Tarih	Onaylayanın Adı Soyadı	İmza ve Mühür
[GG/AA/YYYY]	[SORUMLU AD SOYAD]	

Besin Ayrıntıları ve Kaynak Bilgisinin Gösterilmesi

Bugün kayıtlı besinin ayrıntılarını gösteren pencere üzerinde çalıştım. Benzer adlara sahip besinler; çiğ veya pişmiş olma, kaynak veri seti ve porsiyon seçenekleri bakımından farklı olabilir. Bu nedenle yalnızca besin adını göstermekle yetinmedim. Kullanıcının seçiminin neye dayandığını anlayabilmesi için kaynak bilgilerini, miktarı ve kaydedilmiş besin değerlerini aynı ayrıntı akışında ele aldım.

EntryDetailSheet açılırken foodId üzerinden kaynak ayrıntısı isteniyor. Günlükte tüketilmiş miktara ait değerler ise kaydın kendi nutrients listesinden gösteriliyor. Bu ayrımı korudum; uzak kaynaktaki yüz gramlık değerle kullanıcının gerçekten kaydettiği miktarın değeri birbirine karışmamalı. Besin öğelerini isimlerine göre sıralamak ve uzun içerik için kaydırılabilir pencere kullanmak, listeyi daha rahat okunabilir hâle getiriyor.

Düzenleme işlemini iki farklı ihtiyaca ayırdım. Miktar veya öğün yanlışsa mevcut kaydın porsiyonu değiştiriliyor; besinin kendisi yanlışsa Replace food üzerinden yeniden arama açılıyor. Besin değiştirme sırasında yeni seçimin değerlerini alırken eski kaydın kimliğini ve tarihini korudum. Böylece düzeltme, yanlışlıkla yeni bir öğün eklemek veya eski günü değiştirmek anlamına gelmiyor. Kullanıcı seçimden vazgeçerse özgün kayıt korunuyor.

needsReview işaretli kayıta Quick estimate açıklamasını göstererek tahmini miktarın fark edilmesini sağladım. entry_detail_test.dart içindeki besin değiştirme senaryosu, yeni besin seçildiğinde eski kayıt kimliğinin korunmasını denetliyor. Bu çalışma bana kaynak bilgisini göstermenin yalnızca ayrıntı eklemek olmadığını öğretti: kullanıcı hangi besini kaydettiğini anlayabiliyor ve hatanın miktarda mı, eşleşmede mi olduğunu ayırt ederek doğru düzeltmeyi yapabiliyor.

Çalışılan Tarih	Onaylayanın Adı Soyadı	İmza ve Mühür
[GG/AA/YYYY]	[SORUMLU AD SOYAD]	

Porsiyon ve Yenilebilir Miktar Hesaplamaları

Bugün besin değerlerinin tüketilen miktara göre hesaplanmasını ve yenilebilir ağırlık dönüşümünü çalıştım. Kaynak değerler yüz gram yenilebilir kısım üzerinden verildiğinden temel katsayı $\text{grams} / 100$ şeklindedir. `DiaryEntry.fromFood` bu katsayıyı uygun besin öğelerine uygular. Hesabı ekranın farklı yerlerinde yeniden yazmak yerine modelde tutmak, elle arama ve sesli kayıt yollarının aynı sonucu üretmesini sağlıyor.

`diary_test.dart` dosyasındaki kontrollü elma örneğini kullandım. Test verisinde yüz gram için 52 kcal bulunuyor; 182 gram kaydedildiğinde işlem $52 \times 182 / 100 = 94,64$ kcal oluyor. Protein için $0,26 \times 1,82 = 0,4732$ gram bekleniyor. Bu rakamlar testin hesaplama verileridir. Ondalıklı işlemlerde küçük kayan nokta farklarını karşılamak için eşitliği uygun toleransla karşılaştırmanın neden gerekli olduğunu öğrendim.

Kemik veya kabuk içeren besinlerde satın alınan miktarla yenilen miktarı ayırdım. Testteki 500 gramlık tavuk örneğinde, besine bağlı kullanılabilir katsayı 0,67 olduğu için yenilebilir miktar 335 gramdır. Yüz gramda 161 kcal verisiyle toplam 539,35 kcal hesaplanır. `inputGrams` alanında 500, `grams` alanında 335 tutulması, hem kullanıcının girişini hem hesabın dayanağını koruyor. Benzer isimli başka bir besinin katsayısı kullanılmıyor.

Miktar düzenlemede sonlu, sıfırdan büyük ve en fazla 10.000 gram değer kabul edilmesini inceledim. Virgüllü ondalık girişin noktaya çevrilmesi, Türkçe kullanım için önemlidir. Satın alınan ağırlık seçilmiş ancak uygun katsayı bulunamamışsa kayıt düzeltme gerektiriyor. Gün sonunda doğru formül kadar, formüle verilen ağırlığın neyi temsil ettiğinin de önemli olduğunu gördüm; yanlış ağırlık temeliyle yapılan aritmetik, teknik olarak düzgün görünse bile yanlış sonuç verir.

Çalışılan Tarih	Onaylayanın Adı Soyadı	İmza ve Mühür
[GG/AA/YYYY]	[SORUMLU AD SOYAD]	

Günlük Verilerinin Telefonda Saklanması

Bugün günlüğün uygulama kapatıldıktan sonra korunmasını sağlayan LocalStore üzerinde çalıştım. SharedPreferences içinde kayıt listesi JSON metni olarak saklanıyor. Günlük, hedefler, kişisel profil, izin tercihleri ve günlük öneri için ayrı anahtarlar bulunuyor. Örneğin opennutri.diary.entries.v1 anahtarı öğün listesini temsil ediyor. Anahtarları ayrı tutmak, bir ayarı değiştirirken bütün kullanıcı verisini yeniden biçimlendirme ihtiyacını azaltıyor.

Kaydetme sırasında her DiaryEntry nesnesi toJson ile alanlarına ayrılıyor; oluşan liste jsonEncode ile metne dönüştürülüyor. Açılışta getString ile okunan içerik jsonDecode ve fromJson üzerinden tekrar nesnelere çevriliyor. Henüz kayıt yoksa boş liste dönülüyor. Yalnızca foodId saklamak yerine miktar, tarih ve hesaplanmış besin değerlerini birlikte tutmak, geçmiş öğünlerin kaynak servisine yeniden bağlanmadan gösterilmesini sağlıyor.

saveEntries içindeki setString sonucunu kontrol ettim. Depolama işlemi false dönerse StateError üretiliyor; böylece çağıran ekran başarısız yazmayı tamamlanmış gibi kabul etmiyor. Testlerde gerçek kişisel günlük yerine bellek içi saklama nesneleri ve SharedPreferences test başlangıç değerleri kullanılıyor. JSON dönüşüm senaryosu kimlik, öğün, gram ve kalori bilgisinin geri okunduğunda korunmasını sınavarak veri kaybına karşı somut kontrol sağlıyor.

Bu yöntemin sınırlarını da not ettim. Yerel saklama, bulut eşitleme veya yedekleme anlamına gelmiyor; mevcut uygulamada dışa aktarma ve geri yükleme arayüzü bulunmuyor. Büyük günlükler, bozuk JSON ve uygulamanın beklenmedik kapanması için daha kapsamlı dayanıklılık çalışmaları yapılabilir. Gün sonunda kalıcılığın yalnızca bir kaydetme komutu olmadığını; veri biçimi, hata sonucu ve yeniden açılıştaki okuma davranışının birlikte tasarlanması gerektiğini öğrendim.

Çalışılan Tarih	Onaylayanın Adı Soyadı	İmza ve Mühür
[GG/AA/YYYY]	[SORUMLU AD SOYAD]	

Durum Yönetimi, Hızlı Görünüm ve Geri Alma

Bugün yeni besinin hemen görünmesiyle verinin güvenilir biçimde saklanmasını birlikte ele aldım. ApplicationController kayıt listesini önce bellekte güncelliyor ve ekranlara bildirim gönderiyor. Kullanıcı böylece depolama işleminin bitmesini beklemeden öğününü görebiliyor. Ancak bu iyimser gösterimin, kalıcı yazmanın tamamlandığı anlamına gelmediğini ayırdım. Özellikle hızlı ekleme, düzeltme ve Geri al işlemleri arka arkaya geldiğinde yazma sırası korunmalıdır.

Sorunu somutlaştırmak için önce 100 gramlık kaydın eklendiği, hemen ardından 200 grama düzeltildiği senaryoyu takip ettim. İki yazma bağımsız başlatılırsa eski 100 gramlık liste daha geç tamamlanıp yenisinin üstüne yazılabilir. `_entryWriteTail` ile her yazmayı öncekinin arkasına bağlayan Future zincirini kullandım. `_persistedEntries` ise son başarıyla saklanan listeyi tutuyor. Hatalı yazma hâlâ güncel görünümü temsil ediyorsa arayüz bu son başarılı duruma geri dönüyor.

`addEntries` içinde mevcut kimlikleri bir kümede toplayarak aynı kaydın ikinci kez eklenmesini önledim. Sesli isteğin tekrar işlenmesiyle aynı besinin başka gün yeniden yenmesi farklı durumlardır; ikinci durumda yeni kayıt kimliği üretiliyor. Toplu ekleme ve `removeEntries` ile Geri al işlemi, bütün öğünü tek liste güncellemesi üzerinden yönetiyor. Sayaçlı test nesnesi, iki besinlik grubun tek saklama çağrısıyla yazıldığını kontrol ediyor.

Regresyon testinde ilk yazmayı bekletip düzeltmeyi sıraya aldım; ikinci yazmaya “disk full” hatası verilince kaydın son saklanan 100 grama dönmesi bekleniyor. Bu kontrollü hata, normal bir denemede kolayca görünmeyen yarış durumunu anlamamı sağladı. Gün sonunda hızlı tepki veren arayüzün arkasında işlem sırası, kimlik denetimi ve başarısızlık davranışının birlikte kurulması gerektiğini öğrendim.

Çalışılan Tarih	Onaylayanın Adı Soyadı	İmza ve Mühür
[GG/AA/YYYY]	[SORUMLU AD SOYAD]	

Kayıt Düzenleme ve İptalde Verinin Korunması

Bugün kaydedilmiş bir öğünü düzenlerken özgün verinin kaybolmaması üzerine çalıştım. Özellikle sesli kaydın ardından Edit batch açıldığında mevcut besinleri önce silmek risklidir. Kullanıcı yalnızca kontrol etmek için pencereyi açıp geri dönebilir. Düzenleme ekranının açılmasını veri değişikliği saymak yerine, değişiklikleri ancak Save changes başarıyla tamamlandığında uygulayan akışı kullandım.

Toplu düzenlemede updateEntries, mevcut listedeki kayıtları kimliklerine göre yeni değerlerle değiştiriyor. Editör açılırken veya iptal edilirken silme yapılmıyor. Ayrıca listeden daha önce kaldırılmış bir kaydı, eski bir editörden dönen sonuçla yeniden eklememek için yalnızca mevcut kimlikler güncelleniyor. Böylece “düzenle” ile “yeni kayıt oluştur” işlemleri arasında net bir ayrım bulunuyor.

Tek besinin miktarını değiştiren pencerede withEditedServing yöntemini kullandım. Bu yöntem eski kimliği ve tarihi koruyup besin değerlerini yeni miktarın eski miktara oranıyla ölçekliyor. Örneğin kontrollü testte 100 gramlık, 52 kcal içeren tahmini kayıt 150 grama düzeltildiğinde 78 kcal oluyor. Öğün kahvaltı olarak değiştirilebiliyor ve düzeltme sonrası needsReview işareti kaldırılıyor. Kullanıcının virgüllü girişini kabul etmek için ondalık ayracını da normalleştirdim.

entry_detail_test.dart içindeki iptal, ondalıklı miktar ve besin değiştirme senaryolarıyla işlemleri değerlendirdim. Sesli toplu düzenlemeden çıkma testi de özgün besinlerin günlükte kaldığını kontrol ediyor. Miktar penceresinde metin alanının kapanış animasyonu boyunca geçerli kalmasına dikkat ettim; erken bırakılan denetleyici arayüz hatası oluşturabilir. Bu çalışma, düzenleme başarısını yalnızca Kaydet düğmesine basmakla değil, kullanıcının vazgeçtiği bütün çıkış yollarıyla birlikte değerlendirmeyi öğretti.

Çalışılan Tarih	Onaylayanın Adı Soyadı	İmza ve Mühür
[GG/AA/YYYY]	[SORUMLU AD SOYAD]	

Günlük Enerji ve Makro Besin Toplamları

Bugün öğünlerdeki değerlerin günlük özette doğru toplanması üzerinde çalıştım. DailyTotals sınıfı seçilen güne ait kayıtları alıyor ve besin öğelerini birleştiriyor. Toplamın anahtarı yalnızca görünen isim değil, besin ögesi kimliği ve birimidir. Gramla miligramı veya farklı anlamdaki kaynak alanlarını kontrolsüz biçimde toplamak, ekranda düzgün biçimlendirilmiş fakat yanlış bir sonuç üretebilir.

Enerji hesabında karşılaşılan önemli ayrıntı, USDA kaynaklarının aynı bilgiyi farklı alanlarda taşıyabilmesiydi. Bir besinde Energy, diğerinde Energy (Atwater Specific Factors) bulunabiliyor. Önce bütün alanları topladıktan sonra yalnızca Energy alanını seçmek, diğer besinin kalorisini kaybettiriyordu. Çözümde her DiaryEntry için öncelikle geçerli enerji alanını seçtim; günlük toplamı daha sonra bu tekil kalori değerlerinden oluşturdum.

Regresyon testindeki sayısal örneği adım adım inceledim. İlk kayıta Energy 52 kcal; ikinci kayıta özel Atwater 80 kcal ve genel Atwater 90 kcal bulunuyor. Beklenen sonuç $52 + 80 = 132$ kcal'dir. İkinci besinin iki alternatif enerji alanını birlikte saymak 222 kcal üretir; yalnızca düz Energy alanını almak ise 52 kcal bırakır. Bu örnek, hatanın toplama işleminden çok doğru alanın hangi aşamada seçildiğiyle ilgili olduğunu gösteriyor.

Protein, karbonhidrat ve yağ toplamlarını da seçilen güne ait kayıtlar üzerinden hesapladım. Model testi, 100 ve 50 gramlık kontrollü elma kayıtlarının toplam enerjisinin 78 kcal olduğunu doğruluyor. Kullanıcı başka güne geçtiğinde aynı kayıtların yanlışlıkla toplamda kalmaması gerekiyor. Gün sonunda veri kaynaklarının farklı adlandırmalarını dikkate almadan yapılan toplamamanın eksik veya çift sayım yaratabileceğini öğrendim; testte beklenen sayıyı elle hesaplamak bu tür hataları görünür kılıyor.

Çalışılan Tarih	Onaylayanın Adı Soyadı	İmza ve Mühür
[GG/AA/YYYY]	[SORUMLU AD SOYAD]	

Besin Ögesi Raporu ve Eksik Verinin Yorumlanması

Bugün besin ögesi raporunu ve yapay zekâya gönderilen günlük özeti çalıştım. Enerji ve makro besinlerin yanında vitaminler, mineraller ve lif gibi değerler de gösteriliyor. Burada en önemli konu, kaynakta bulunmayan bir değerle gerçekten sıfır ölçülmüş bir değeri ayırmaktı. Bir besinin D vitamini bilgisi yoksa bunu “D vitamini tüketilmedi” diye yorumlamak doğru değildir.

CoachService içindeki _nutrientMetric yönteminde önce her kayıta ilgili ad ve birime sahip alanı aradım. Alan bulunan kayıt sayısı ayrıca tutuluyor; hiç değer bulunmazsa amount null gönderiliyor. Değer varsa bilinen miktarlar toplanıyor ve toplam kaç besinden kaçında veri bulunduğu bağlama ekleniyor. Böylece model yalnızca miktarı değil, o miktarın eksik kapsama dayanıp dayanmadığını da görebiliyor.

Testte üç günlük kayıt kullanılıyor: birinde 5 mikrogram D vitamini, ikincisinde sıfır, üçüncüsünde ise hiç alan bulunmuyor. Beklenen toplam 5, bilinen kayıt sayısı 2 ve toplam kayıt sayısı 3. Aynı örnekte kalsiyum bilgisi hiç yoksa null kalıyor. Mikrogram için µg, ug ve mcg gibi farklı yazımların aynı birime dönüştürülmesi de kontrol ediliyor. Sıfırın veri olarak korunması, eksik bilgiyle karıştırılmaması açısından önemliydi.

Raporun açıklamalarında mevcut günlük değerlerini kişisel tıbbi tanı gibi sunmamaya dikkat ettim. Uygulamada kullanılan genel referanslarla kişinin elle belirlediği hedefleri ayırdım; düşük görünen bir günlük toplam tek başına beslenme yetersizliği kanıtı değildir. Bu çalışma bana raporlama kalitesinin yalnızca çok sayıda gösterge eklemek olmadığını öğretti. Bilginin kaynağını, birimini ve eksik kalan kısmını açıklamak, sonucun kullanıcı tarafından doğru anlaşılması için gereklidir.

Çalışılan Tarih	Onaylayanın Adı Soyadı	İmza ve Mühür
[GG/AA/YYYY]	[SORUMLU AD SOYAD]	

Kişisel Hedefler ve Ayar Formlarının Tutarlılığı

Bugün kullanıcının enerji, protein, karbonhidrat ve yağ hedeflerini değiştirdiği ayarlar üzerinde çalıştım. NutritionTargets bu değerleri ortak modelde tutuyor; günlüğün ilerleme göstergeleri ve öneri bağlamı aynı hedefleri kullanıyor. Formun yalnızca sayı kabul etmesi yeterli değildi. Sıfır, negatif veya sonlu olmayan değerleri engellemek, hem anlamsız hedefleri hem de oran hesaplamalarında oluşabilecek sorunları önlemek için gerekiyordu.

AppController.updateTargets içinde bütün hedefleri doğrulayıp sonrasında ortak durumu güncelledim. Yeni değerlerle birlikte günlük öneri önbellediği de temizleniyor. Eski hedefe göre hazırlanmış bir önerinin, yeni hedef seçildiğinde güncelmiş gibi görünmesini istemedim. Değişikliğin yalnızca bellekte değil yerel saklamada da uygulanması, uygulama yeniden açıldığında eski önerinin geri gelmesini önüyor.

Ayar formunda daha ince bir eşitleme sorununu ele aldım. Diyet değişirse hedef alanlarının yeni değerleri göstermesi gerekiyor; fakat kullanıcı bir alana henüz kaydetmediği sayı yazarken ilgisiz günlük değişikliği bu metni ezmemeli. Bu nedenle formu her denetleyici bildiriminde baştan doldurmak yerine, ilgili hedef veya profil değişikliğini ayırt eden davranışı kullandım. Kullanıcının yazdığı geçici metinle kaydedilmiş hedef farklı durumlar olarak ele alındı.

Regresyon testi, diyet değişince formun güncellenmesini ve ilgisiz bildirimde kaydedilmemiş metnin korunmasını birlikte denetliyor. Ayrıca kişisel hedef değişiminin günlük öneriyi yeniden yüklenen denetleyicide de geçersiz bırakması sınanıyor. Gün sonunda durum yönetiminin yalnızca ortak değerleri paylaşmak olmadığını gördüm. Bir ekranın geçici kullanıcı girdisini korurken dışarıdan gelen gerçek değişikliklere uyum sağlaması, doğru form davranışının önemli bir parçasıdır.

Çalışılan Tarih	Onaylayanın Adı Soyadı	İmza ve Mühür
[GG/AA/YYYY]	[SORUMLU AD SOYAD]	

Mikrofon İzni ve Ses Kaydının Başlatılması

Bugün sesli besin kaydının telefon tarafındaki başlangıcını geliştirdim. Mikrofon işlemlerini doğrudan ekranın içine yerleştirmek yerine VoiceRecorderSession arayüzü üzerinden yönettim. Bu arayüz başlatma, durdurma, iptal ve geçici dosyayı silme işlemlerini tanımlıyor. OpenNutriVoiceRecorder gerçek mikrofonu kullanırken testlerde aynı arayüzü uygulayan sahte kayıt nesnesi verilebiliyor. Böylece ekran davranışını sınamak için her seferinde gerçek ses kaydetmek gerekmiyor.

Kayıt başlamadan önce record paketinin izin kontrolünü kullandım. İzin verilmemişse durum permissionDenied olarak değişiyor ve kullanıcıya mikrofon izni gerektiği açıklanıyor. Bu hata, besin servisinin çalışmamasıyla aynı mesaj altında gösterilmiyor. Kullanıcı sesli kaydı kullanmak istemezse elle aramaya geçebiliyor. İzin reddini, uygulamanın bütün günlük işlevlerini durduran bir durum hâline getirmemeye dikkat ettim.

Ses dosyasını uygulamanın geçici dizininde, zaman bilgisinden üretilmiş bir adla oluşturdum. Kayıt ayarları WAV biçimi, 16.000 Hz örnekleme ve tek kanaldır. Bu seçim, konuşma için sınırlı büyüklükte ve servis tarafından işlenebilir bir dosya hazırlıyor. Ses seviyesini 100 milisaniyede bir dinleyerek kayıt ekranının güncellenmesini sağladım; bu seviye bilgisi bir sonraki aşamadaki sessizlik algılamasında da kullanılıyor.

Kayıt durumlarını boшта, kaydediyor, durdu, izin reddedildi ve hata şeklinde ayırdım. Kayıt başlatılamadığında geçici dosya temizleniyor ve ekran uygun duruma dönüyor. Dosya tamamlandıktan sonra çözümleme isteğine geçiliyor; uygulama sürekli açık mikrofonla arka planda konuşma göndermiyor. Gün sonunda bir cihaz özelliğini eklerken yalnızca başarılı başlatmayı değil, izinleri, ekran durumunu, dosya yaşam döngüsünü ve test edilebilirliği birlikte düşünmem gerektiğini öğrendim.

Çalışılan Tarih	Onaylayanın Adı Soyadı	İmza ve Mühür
[GG/AA/YYYY]	[SORUMLU AD SOYAD]	

Sessizlik Algılama ve Geçici Ses Dosyalarının Yönetimi

Bugün kullanıcının konuşmayı bitirdiğinde kaydın otomatik durmasını sağlayan mantık üzerinde çalıştım. Amaç, kişinin her seferinde durdurma düğmesini aramak zorunda kalmamasıydı. SilenceStopDetector ses seviyesini geçen süreyle birlikte değerlendiriyor. Başlangıçtaki kısa duraksamanın kayıt sonu sanılmaması için ilk 800 milisaniyede sessizlik nedeniyle durdurma kararı verilmiyor.

Gerçek kayıt sınıfında eşik -40 dBFS, konuşma sonrasındaki sessizlik süresi 1,6 saniye olarak ayarlanmış. Eşiğin üstünde ses algılandığında sessizliğin başlangıcı sıfırlanıyor; düşük seviye yeterince uzun sürerse stop çağrılıyor. Buna ek olarak 30 saniyelik kesin süre sınırı bulunuyor. Böylece gürültülü ortamda sessizlik algılanmasa bile tek kaydın süresi sınırsız büyümüyor. Bu eşiklerin basit ses seviyesi mantığı olduğunu, kusursuz konuşma tanıma anlamına gelmediğini öğrendim.

Durdurma işleminin hem düğmeden hem zamanlayıcıdan tetiklenebilmesini dikkate aldım. _stopping işareti aynı dosyanın iki kez durdurulmasını sınırlıyor. Kayıt bittiğinde ses seviyesi aboneliği ve zamanlayıcı kapatılıyor; iptal veya işleme sonrasında geçici dosya siliniyor. Kaynakları yalnızca başarılı yolda temizlemek yeterli olmadığından hata ve iptal yollarını da takip ettim. İşletim sistemi süreci aniden sonlandırırsa bu temizliğin garantili olmadığını ayrıca not ettim.

Birim testinde gerçek mikrofon yerine belirli zamanlarda verilen -60 ve -20 dBFS değerleri kullanılıyor. Test, başlangıç fırsatını ve yeniden yükselen sesin sessizlik sayacını sıfırlamasını kontrol ediyor. Test nesnesinin varsayılan son sessizliği 2 saniyedir; gerçek kaydedici bunu 1,6 saniyeye ayarlar. Bu ayrımı görmek, testteki sayıyı doğrudan çalışma ayarı sanmamayı öğretti. Gün sonunda zamanlayıcılarla çalışan bir özellikte süre, eşik ve kaynak temizliğinin birlikte değerlendirilmesi gerektiğini anladım.

Çalışılan Tarih	Onaylayanın Adı Soyadı	İmza ve Mühür
[GG/AA/YYYY]	[SORUMLU AD SOYAD]	

Ses Servisine Güvenli İstek Gönderilmesi

Bugün tamamlanmış ses dosyasını mevcut çözümleme servisine gönderen mobil istemciyi çalıştım. VoiceApiClient, doğrudan Gemini'ye gizli anahtarla bağlanmak yerine uygulamaya ayrılmış kimlik doğrulama oturumunu kullanıyor. Oturum yoksa anonim giriş yapılıyor ve erişim belirteci Authorization: Bearer başlığına ekleniyor. Buradaki anonimliğin ses içeriğinde kişisel bilgi bulunamayacağı anlamına gelmediğini, yalnızca kullanıcıya ayrı bir hesap formu açılmadığını ayırdım.

POST /v1/voice/resolve isteğini multipart biçiminde hazırladım. Dosya audio alanına meal.wav adı ve audio/wav içerik türüyle ekleniyor. language_hint, local_timestamp ve timezone alanları konuşmanın dili ve zaman bağlamını taşıyor. İngilizce ve Türkçe cihaz yerelleri desteklenen ipuçlarına dönüştürülüyor; diğerlerinde otomatik dil seçimi isteniyor. Dosya 1 MB sınırını aşıyorsa gönderimden önce hata veriliyor.

Beklemeyi azaltmak için kayıt başlarken oturum ve servis hazırlığının da başlatılmasını inceledim. warmUp başarısız olursa mikrofon kaydı engellenmiyor; asıl isteğin hata yönetimi daha sonra devreye giriyor. Aynı anda birden fazla oturum isteği oluşmasını önlemek için devam eden belirteç isteği paylaşılabiliyor. Belirteç hazırlığına 10 saniye, gönderilen ses isteğinin yanıtını bütünüyle almaya 30 saniye sınır uygulanıyor.

Yanıttan VoiceResolution modeli üretilmesini ve ağ, zaman aşımı, kimlik doğrulama ile beklenmeyen yanıt hatalarının ayrılmasını ele aldım. Sadece HTTP başlığının gelmesi tamamlanma sayılmıyor; yanıt gövdesinin de alınması gerekiyor. Bu çalışma mevcut servisin mobil entegrasyonudur; sunucu altyapısını sıfırdan geliştirme çalışması değildir. Gün sonunda güvenli API kullanımında anahtarların konumu, dosya boyutu, istek bağlamı ve bekleme sınırlarının birlikte düşünülmesi gerektiğini öğrendim.

Çalışılan Tarih	Onaylayanın Adı Soyadı	İmza ve Mühür
[GG/AA/YYYY]	[SORUMLU AD SOYAD]	

Ses Çözümleme Sonucunun Besin Adaylarına Dönüştürülmesi

Bugün ses servisinin döndürdüğü yanıtı günlükte kullanılabilir besinlere dönüştürdüm. Yanıtta duyulan metin, çözümlenen besinler, aday eşleşmeler, miktar bilgisi ve istek kimliği bulunuyor. Sesin metne doğru çevrilmesiyle doğru besinin seçilmesinin farklı aşamalar olduğunu gördüm. Örneğin bir ifade anlaşılmış olsa bile hazırlanma biçimi veya miktarı belirsiz kalabilir; bu durumda metni kaybetmeden kullanıcıya düzeltme yolu sunmak gerekiyor.

VoiceResolution ve ilgili alt modeller üzerinden seçilmiş adayla alternatifleri ayırdım. Uygulama besin değerlerini modelin metninden üretmiyor; adayın foodId değeriyle Core ayrıntısını yükleyip kaynak besin öğelerini alıyor. Böylece dil çözümleme sonucu, hesaplama verisinin yerine geçmiyor. Duyulan ifadeyi ve seçilen besini birlikte göstermek, kullanıcıya eşleşmenin doğru olup olmadığını kontrol etme imkânı sağlıyor.

Hızlı akışta önce yalnızca seçilmiş besinlerin ayrıntıları yükleniyor. Kimlikler bir kümeye alınarak aynı besin için tekrarlı ayrıntı isteği azaltılıyor; farklı ayrıntılar Future.wait ile birlikte bekleniyor. Her şey kullanılabilir durumdaysa bütün alternatifleri indirmeden kayda geçiliyor. İnceleme gerektiğinde aday listesi genişletiliyor. Bu düzen, kullanıcının ihtiyaç duymadığı seçenekleri bekleme süresine eklememeyi sağlıyor.

Besin ayrıntısı yüklenemezse konuşma sonucunun tamamen başarısız sayılmamasına dikkat ettim. Ekran, öğünün anlaşıldığını fakat sayısal ayrıntıların yüklenemediğini açıklayabiliyor; yeniden deneme veya elle arama sunuyor. Geçersiz yapılandırılmış yanıt senaryosu da korunabilen konuşma metninin ekranda kalmasını denetliyor. Gün sonunda çok aşamalı bir işlemde her aşamanın sonucunu ayrı tutmanın önemini öğrendim: son adımın hatası, önceki adımda elde edilmiş yararlı bilgiyi silmemelidir.

Çalışılan Tarih	Onaylayanın Adı Soyadı	İmza ve Mühür
[GG/AA/YYYY]	[SORUMLU AD SOYAD]	

Doğrudan Kayıt ve Sonradan Düzeltme Akışı

Bugün sesli kaydın her başarılı çözümlemeden sonra onay formunda beklemesini kaldıran akış üzerinde çalıştım. Kullanıcının amacı öğünü hızlı eklemek olduğu için, kullanılabilir besinler doğrudan günlüğe yazılıyor. _canInstantLog, grubun boş olmamasını ve bütün öğelerin geçerli olmasını esas alıyor. Burada “geçerli”, seçilmiş Core ayrıntısının, uygun miktarın ve kullanılabilir ağırlık temelini bulunması anlamına geliyor.

Miktar söylenmemişse 100 gram, ağırlık temeli belirtilmemişse yenilebilir ağırlık varsayılanını kullandım. Bu değerleri ölçülmüş bilgi gibi sunmadım; belirsiz eşleşmeler ve varsayılanlar needsReview üzerinden Quick estimate olarak işaretleniyor. Kullanıcı açıkça satın alınan ağırlık belirtmiş fakat seçilen besinde kullanılabilir dönüşüm katsayısı yoksa kayıt engelleniyor. Bir öğe bile kullanılabilir değilse mevcut uygulama bütün grubu incelemeye bırakıyor; kısmi kaydetme kuyruğu bulunmuyor.

Kayıt kimliğini servis requestId değeriyle besinin conceptIndex bilgisini birleştirerek oluşturdum. Aynı yanıt yeniden işlendiğinde aynı kimlik üretilmesi, denetleyicideki tekrar denetiminin çift kayıt oluşmasını önlemesine yardımcı oluyor. Kullanıcı isterse Undo ile grubu kaldırabiliyor veya Edit batch ile değiştirebiliyor. İsteğe bağlı geri bildirim, yerel kayıt tamamlandıktan sonra beklenmeden gönderiliyor; bu ek servis başarısız olsa da kayıt başarısı geri alınmıyor.

Sesli kayıt testlerinde miktarsız yanıtın tahmin olarak eklenmesi, verilen gram miktarının korunması ve geri bildirimin başarı ekranını geciktirmemesi denetleniyor. Düzenleme iptal edildiğinde özgün kayıtların korunması da aynı akışın devamıdır. Gün sonunda hızlı kayıtların doğruluğun birlikte yönetilebileceğini gördüm: kullanılabilir bilgiyi hemen kaydetmek, belirsizliği açık göstermek ve düzeltmeyi kolaylaştırmak, bütün durumlarda onay bekletmekten farklı bir kullanıcı deneyimi sağlıyor.

Çalışılan Tarih	Onaylayanın Adı Soyadı	İmza ve Mühür
[GG/AA/YYYY]	[SORUMLU AD SOYAD]	

Bekleme Süreleri ve Servis Hatalarının Ayırıştırılması

Bugün sesli kayıt ve yapay zekâ ekranlarında beklemenin hangi aşamalardan oluştuğunu araştırdım. Kullanıcının gördüğü toplam süre; kaydın bitmesi, oturum hazırlanması, istek kuyruğu, ses yükleme, model yanıtı ve besin ayrıntılarını alma işlemlerini içerebiliyor. Bu nedenle tek bir zaman aşımı değerini bütün akışın kesin süresi gibi yorumlamadım. Mikrofon açıkken hazırlık yapmak ve yalnızca gerekli ayrıntıları yüklemek, farklı bekleme kaynaklarını azaltan uygulama kararlarıdır.

Sunucu kullanıcı başına tek etkin yapay zekâ isteğine izin verdiğiinden mobilde ortak bir istek sırası kullandım. VoiceApiClient içindeki _serialAi, günlük öneri, Oracle, metin çözümleme ve ses işlemlerini aynı Future zincirine bağlıyor. Bir isteğin hatası zinciri kalıcı olarak durdurmamalı; sonraki işlem çalışabilmelidir. Ekranda henüz açılmamış Oracle için erken istek oluşturmamak da bu sırayı gereksiz doldurmamayı sağlıyor.

Hata açıklamalarında bağlantı sorunu, zaman aşımı, kimlik doğrulama, kota ve geçersiz yanıtı ayırdım. Özellikle HTTP 429, servise ulaşamadığını değil istek sınırına takıldığını gösterebilir. Kullanıcıya her durumda “Gemini kullanılamıyor” demek çözüm yolunu belirsizleştiriyordu. Servisin yapılandırılmış tek yedek model denemesinden gelen yanıtta gerçek model adının taşınmasını da kullandım; ekranda her başarılı yanıtı ana model üretmiş gibi göstermedim.

Testte ilk yapay zekâ isteğini bekletip ikinciye sıraya aldım. İlk yanıt 503 hatasıyla tamamlandığında ikinci isteğin başlayıp yanıt döndürmesi bekleniyor. Oracle ekranındaki ayrı test ise 429 durumunda bağlantı hatası yerine istek sınırı açıklamasını arıyor. Gün sonunda performans ve hata yönetiminin ilişkisini daha iyi anladım: gereksiz istekleri azaltmak kadar, bekleyen veya başarısız işlemin nedenini doğru sınıflandırmak da kullanıcının ne yapacağını anlamasını sağlıyor.

Çalışılan Tarih	Onaylayanın Adı Soyadı	İmza ve Mühür
[GG/AA/YYYY]	[SORUMLU AD SOYAD]	

Android Ana Ekran Araç Takımının Geliştirilmesi

Bugün telefonun ana ekranındaki mikrofon araç takımını Flutter uygulamasına bağlayan Android bölümünü çalıştım. Araç takımı uygulama ekranının dışında yaşadığı için yalnızca bir Flutter düğmesi eklemek yeterli değildi. Kotlin tarafındaki VoiceLogWidgetProvider, Android'in AppWidgetProvider yapısını kullanıyor. Görünüm RemoteViews ile hazırlanıyor ve mikrofon düğmesine MainActivity'yi hedefleyen bir PendingIntent atanıyor.

Tıklama isteğine ACTION_VOICE_LOG eylemini ekledim. MainActivity bu eylemi yakalayıp bekleyen sesli kayıt isteği olarak tutuyor. Uygulama kapalıyken açılmasıyla zaten çalışırken yeni istek alması farklı giriş yollarıdır: ilk durumda başlangıç Intent'i, ikinci durumda onNewIntent kullanılıyor. Aynı eylemin tekrar işlenmemesi için yakalandıktan sonra Intent üzerindeki action temizleniyor; bekleyen işaret tüketildiğinde de sıfırlanıyor.

Kotlin ile Dart arasındaki haberleşmeyi MethodChannel üzerinden kurdum. org.opennutri.app/voice_widget kanalında Flutter, consumePendingVoiceAction ile bekleyen işlemi alabiliyor; çalışan uygulamaya voiceLogRequested bildirimi gönderilebiliyor. Flutter tarafı mevcut VoiceLogScreen ekranını hızlı kayıt modunda açıyor. Böylece araç takımına özgü ikinci bir kayıt sistemi oluşturmak yerine aynı ses, eşleştirme ve saklama akışı yeniden kullanılıyor.

Native testlerde başlangıç ve yeni Intent yollarının eylemi bir kez tüketmesi denetleniyor. Bu testin telefonun ana ekranına gerçekten araç takımı yerleştirildiğini kanıtlamadığını ayrıca ayırdım. Gün sonunda Flutter ile platform kodunun görev paylaşımını öğrendim: Android ana ekran bileşenini ve açılış isteğini yönetirken, uygulamanın ortak kayıt mantığı Dart tarafında kalıyor. Bir kısa yolun güvenilir olması için sadece ilk açılışın değil, tekrar açılma ve yinelenen istek durumlarının da düşünülmesi gerekiyor.

Tablo 2. Araç takımından kayıt ekranına geçiş

1. Araç takımı	Mikrofon düğmesi, sesli kayıt Intent işlemini başlatır.
2. MainActivity	İstek yakalanır ve bir kez tüketilecek biçimde tutulur.
3. MethodChannel	Flutter mevcut sesli kayıt ekranını açar.

Çalışılan Tarih	Onaylayanın Adı Soyadı	İmza ve Mühür
[GG/AA/YYYY]	[SORUMLU AD SOYAD]	

Araç Takımını Ekleme ve Kayıt Sonrası Kapanış

Bugün araç takımının kullanıcı tarafından bulunmasını ve kayıt sonrası telefonun ana ekranına dönüşünü ele aldım. Uygulamanın kurulmuş olması mikrofonun kendiliğinden ana ekrana yerleştiği anlamına gelmiyor. Ayarlar ekranının üst bölümüne Add microphone widget girişini koyan akışı kullandım. Android tarafında sürüm ve başlatıcının pinleme desteği kontrol edilerek requestPinAppWidget çağrılıyor; yerleştirme için kullanıcının onayı gerekiyor.

İstek kabul edilse bile kullanıcının ekleme penceresini iptal edebileceğini dikkate aldım. Bu yüzden yöntemden dönen true değerini “araç takımı eklendi” diye yorumlamadım. Destek yoksa ana ekranda boş alana uzun basma, Widgets bölümünü açma ve OpenNutri mikrofonunu seçme adımları açıklanıyor. Kurulum testleri desteklenen ve desteklenmeyen yanıtları taklit ederek uygulamanın uygun yönlendirmeyi gösterdiğini denetliyor.

Hızlı kayıt açılırken günlük tarihini bugüne aldım. Uygulama daha önce geçmiş bir günü gösteriyor olsa da ana ekran mikrofonundan söylenen yeni öğün o eski güne yazılmamalı. Kayıt tamamlandığında Flutter önce addEntries sonucunu bekliyor, ardından kısa başarı görünümünden sonra finishQuickCapture çağırıyor. Kotlin tarafı eklenen besin sayısını Toast ile gösterip finishAndRemoveTask üzerinden uygulama görevini kapatıyor.

Testte yerel saklamayı yapay olarak bekleterek kapanış çağrısının erken gelmediğini kontrol ettim. Saklama tamamlanınca kapanışa izin verilmesi, “söyle ve dön” akışının veri kaybetmeden tamamlanmasını sağlıyor. Bu özellik görünür kayıt ekranını kullanıyor; kullanıcı işlem bitmeden kapatırsa arka planda garantili devam eden kuyruk yok. Gün sonunda küçük görünen bir araç takımında bile kurulum, tarih, kalıcılık ve kapanış sırasının ayrı ayrı doğrulanması gerektiğini öğrendim.

Çalışılan Tarih	Onaylayanın Adı Soyadı	İmza ve Mühür
[GG/AA/YYYY]	[SORUMLU AD SOYAD]	

Beslenme Şablonları ve Hedeflere Uyarlama

Bugün kişisel hedeflerle birlikte kullanılabilen diyet şablonları üzerinde çalıştım. Uygulamada dengeli, Akdeniz, yüksek proteinli, bitki ağırlıklı, düşük karbonhidratlı keto ve Blue Zones esinli altı seçenek bulunuyor. DietPreset sınıfında ad, açıklama, temel ilkeler ve karbonhidrat/protein/yağ enerji payları tutuluyor. Şablonu yalnızca görsel bir kart olarak değil, hedef hesaplamasına bağlanan veri olarak ele aldım.

targetsForCalories yöntemi kullanıcının mevcut kalori hedefini koruyup makro hedeflerini hesaplıyor. Protein ve karbonhidrat payları 4'e, yağ payı 9'a bölünerek gram değerine çevriliyor. Kodun dengeli şablonunu açıklamak için 2.000 kcal örneğini kullandım: yüzde 25 protein 125 gram, yüzde 45 karbonhidrat 225 gram, yüzde 30 yağ yaklaşık 66,7 gram verir. Bu, şablon hesabını gösteren örnektir; kişiye özel tıbbi öneri değildir.

Hedefe göre yapılan küçük oran değişikliklerini de inceledim. Kas geliştirme veya yağ kaybı hedefinde, karbonhidrat payı uygun düzeydeyse beş yüzde puanı proteine aktarılıyor. Dengeli örnekte böylece protein 150, karbonhidrat 200 gram olurken toplam enerji korunuyor. Performans hedefinde keto dışındaki uygun şablonlarda yağdan karbonhidrata aktarım yapılıyor. Kullanıcı bu başlangıç değerlerini daha sonra ayarlardan değiştirebiliyor ve serbest açıklama ekleyebiliyor.

personalization_test.dart içindeki senaryo şablonun hedefe uyarlanmasını ve sonradan düzenlenebilmesini kontrol ediyor. Şablon değişikliği, eski günlük önerinin temizlenmesini de tetikliyor. Bu çalışmada otomatik kalori ihtiyacı hesabı veya haftalık menü üretildiğini varsaymadım; mevcut özellik makro oranları ve öneri bağlamını düzenliyor. Gün sonunda kişiselleştirmenin sabit bir listeyi kullanıcıya göstermenin ötesinde, seçimin hesaplamalara ve sonraki ekranlara tutarlı yansımaları gerektirdiğini öğrendim.

Çalışılan Tarih	Onaylayanın Adı Soyadı	İmza ve Mühür
[GG/AA/YYYY]	[SORUMLU AD SOYAD]	

Günlük Yapay Zekâ Önerileri ve Önbellek Yönetimi

Bugün uygulama açıldığında gösterilen günlük öneri kartı üzerinde çalıştım. CoachService, kullanıcının seçtiği tarih, hedef, diyet, notlar, açıkça kaydedilmiş tercihleri ve günlük toplamlarından sınırlı bir istek hazırlıyor. Seçilen gündeki son besinler de bağlama ekleniyor; bütün günlük geçmişi gönderilmiyor. Bu düzen, modelin kişiye ve güne uygun öneri verebilmesi için gerekli bilgiyi sınırlı tutmayı amaçlıyor.

DailyCoachBrief öneriyi dateKey ile birlikte yerelde saklıyor. Aynı gün için geçerli öneri varsa her ekran açılışında yeniden model çağırısı yapılmıyor. Hedef, diyet veya kayıtlı tercih değiştiğinde önbellek hem bellekten hem diskten temizleniyor. Başka güne geçildiğinde önceki günün kartı gösterilmiyor. Günlükteki her besin düzeltmesinin otomatik yeni çağrı üretmediğini, kartın belirli anda hazırlanmış bir özet olduğunu ayırdım.

İstek devam ederken kullanıcı tarih veya profil değiştirebilir. Bu yüzden istek başlangıcındaki coachContextRevision değeri yanıt geldiğinde tekrar karşılaştırılıyor. Bağlam değişmişse eski yanıt kaydedilmiyor; uygun ekranda güncel bağlamla yenileme yapılabilir. Servis kullanılmadığında yerel hedef ilerlemesine dayalı basit öneri hazırlanıyor ve generatedByAi false olarak işaretleniyor. Yerel metni yeni üretilmiş yapay zekâ yanıtı gibi göstermedim.

Önbellek testinde önce öneriyi saklayıp diyet, hedef veya tercih değişikliği uyguladım; ardından yeni denetleyiciyle veriyi tekrar yükleyen senaryoyu değerlendirdim. Eski önerinin null gelmesi, temizliğin yalnızca ekranda yapılmadığını doğruluyor. Tarih testi de kartın yanlış gün altında görünmemesini denetliyor. Gün sonunda önbelleğin yalnızca istek sayısını azaltmak olmadığını öğrendim: hangi bilginin ne zaman geçersiz sayılacağını doğru belirlemek, önerinin anlamını ve güvenilirliğini koruyor.

Çalışılan Tarih	Onaylayanın Adı Soyadı	İmza ve Mühür
[GG/AA/YYYY]	[SORUMLU AD SOYAD]	

Yazılı ve Sesli Koç Görüşmesi, Tercihlerin Saklanması

Bugün kullanıcının koça yazılı veya sesli mesaj gönderebildiği görüşme ekranını çalıştım. Metin gönderiminde baştaki ve sondaki boşlukları temizledim; boş, fazla uzun veya başka gönderim sürerken oluşturulan isteği engelleyen kontrolleri kullandım. Mesaj kutusunu gönderim sırasında temizlemek arayüzü sadeleştiriyor; istek başarısız olursa kullanıcının yeniden yazmak zorunda kalmaması için metin kutuya geri konuluyor.

Görüşme ile kalıcı tercih belleğini ayrı tuttum. Ekrandaki konuşma geçmişi oturumluktur ve isteğe en fazla son altı mesaj bağlam olarak aktarılıyor. CoachMemory ise açık kullanıcı ifadelerinden çıkarılan kısa bilgileri saklıyor. Örneğin “Et yemiyorum” ifadesi bir tercih adayı olabilir; yalnızca belirli bir besin hakkında soru sormak, kullanıcının onu tükettiğini veya ona alerjisi olduğunu kanıtlamaz. Bu örneği gerçek kullanıcı bilgisi değil, ayrımın açıklaması olarak kullandım.

Servisten dönen memoryUpdates alanını profil yönetimine bağladım. Aynı bilginin tekrarlanmasını sınırlayan denetim bulunuyor ve en fazla 30 tercih saklanıyor. Eklenen bilgiler yanıtta gösteriliyor; ayarlardan tek tek silinebiliyor. Silme, sonraki önerilerde gönderilen bağlamı da değiştiriyor. Yalnızca açık ifadeleri kaydetme kuralının model talimatına dayandığını, dilin anlamını kusursuz doğrulayan ayrı bir güvence olmadığını not ettim.

Sesli görüşmede aynı sınırlı kayıt bileşeninden yararlanılıyor, fakat amaç besini günlüğe eklemek değil koça mesaj göndermektir. Geçici dosya işlem sonunda temizleniyor; yanıtta varsa konuşma metni ve cevaplayan model gösteriliyor. Profil testleri saklama ve silme davranışlarını değerlendirmeye yardımcı oluyor. Gün sonunda kişiselleştirmede çok veri toplamının değil, kullanıcı tarafından ifade edilen yararlı bilgiyi görünür ve yönetilebilir biçimde saklamanın önemli olduğunu öğrendim.

Çalışılan Tarih	Onaylayanın Adı Soyadı	İmza ve Mühür
[GG/AA/YYYY]	[SORUMLU AD SOYAD]	

Oracle Ekranı ve Öneriden Besin Aramasına Geçiş

Bugün kullanıcının hedeflerine göre besin fikirleri sunan Oracle ekranı üzerinde çalıştım. Bu ekran, günlükteki bilinen toplamaları ve kişisel tercihleri koç servisine oracle modu ile gönderiyor. Yanıtın amacı bir sonraki öğünde düşünülebilecek besinleri ve gerekçelerini sunmak. Öneriyi doğrudan günlük kaydı saymadım; kullanıcı kaynak besini ve porsiyonu seçmeden sayısal beslenme hesabına eklenmiyor.

Oracle'ı yalnızca kullanıcı ilgili sekmeyi açtığında yükleyen davranışı kullandım. Uygulama açılır açılmaz görünmeyen bu ekran için çağrı yapmak, günlük öneri veya sesli kayıtla aynı istek sırasını gereksiz doldurabiliyordu. İçerik değişikliklerini izleyip görünür ekran açıldığında yenileme yapılmasını sağladım. İstek başlangıcındaki bağlam değişmişse eski yanıtın güncel öneri gibi kullanılmaması için durum denetimlerini takip ettim.

Önerideki eylem bir besin kimliği veya modelin uydurduğu besin değerleri yerine, İngilizce Core arama sorgusu taşıyor. Kullanıcı eyleme dokununca bu sorguyla FoodSearchScreen açılıyor; gerçek besin ayrıntısı yükleniyor ve porsiyon seçiliyor. Bu bağlantı, öneri üretme ile besin doğrulamasını birbirinden ayırıyor. Model önerisinin makul görünmesi, veri kaynağında doğru ürüne karşılık geldiğini tek başına kanıtlamaz.

Testlerde Oracle'ın görünmeden istek başlatmaması ve kota hatasını bağlantı kesintisinden ayırması ele alınıyor. Bu özellik için matematiksel olarak en iyi beslenmeyi hesapladığı iddiasında bulunmadım; mevcut yapı bağlama dayalı besin fikirleri sunuyor. Alerji ve özel kısıtların doğrulanması da yalnızca öneri metnine bırakılamaz. Gün sonunda yapay zekâ özelliğini kullanılabilir kılan şeyin sadece cevap üretmek olmadığını gördüm; cevabın kontrollü bir kullanıcı işlemine ve doğrulanmış veri kaynağına bağlanması gerekiyor.

Çalışılan Tarih	Onaylayanın Adı Soyadı	İmza ve Mühür
[GG/AA/YYYY]	[SORUMLU AD SOYAD]	

Kullanıcı Bilgilendirmesi ve Ayarlar Ekranındaki Hatalar

Bugün sesli kayıt, koç ve geri bildirim seçeneklerinin kullanıcıya nasıl açıklandığını çalıştım. Telefonun mikrofon izniyle yapay zekâ servisine veri gönderme açıklaması farklı ihtiyaçlardır. Koç önerilerini kabul etmek de sesli besin kaydını kullanmanın zorunlu koşulu değildir. Bu nedenle özelliklerin izinlerini ve tercihlerini ayrı tuttum; isteğe bağlı geri bildirimin kapalı kalması temel kaydı engellemiyor.

Koç açıklamasında hangi günlük özetinin ve tercihlerin hizmete gönderildiğini, kullanılan sağlayıcıya ilişkin veri işleme bilgisini görünür tuttum. Açıklama kapsamı değiştiğinde eski onayın otomatik geçerli sayılmaması için sürüm bilgisini kullandım. Regresyon testinde yalnızca eski coach_enabled alanına sahip profilin yeniden onay gerektirmesi denetleniyor. Kullanıcının ayarlardan koçu kapatabilmesi ve saklanan bilgileri silebilmesi, açıklamanın yanında gerçek kontrol de sunuyor.

Ayarlar ekranında servis durumu satırı görünür alanın altında kaldığında oluşabilen hatayı ele aldım. İstek başlamış olsa da ListView içindeki FutureBuilder henüz oluşturulmamış olabilir. İstek o anda başarısız olursa hatayı dinleyen bileşen bulunmayabilir. _checkCoreHealth içinde request.ignore() ile hatanın baştan ele alınmasını sağladım; aynı özgün Future döndürüldüğü için satır görünür olduğunda başarısız durum yine gösteriliyor. Bu yöntem hatayı kullanıcıdan saklamak değil, erken gelen hatanın yakalanmadan kalmasını önlemektir.

Testte çevrimdışı servis nesnesiyle ayarlar açılıyor; önce yakalanmamış hata oluşmadığı, ardından aşağı kaydırınca Unavailable yazısının görüldüğü kontrol ediliyor. İsteğe bağlı geri bildirim ve eski onayın yenilenmesi de ayrı senaryolardır. Bu çalışma, ekrandaki bir satırın henüz çizilmemiş olmasının ağ isteği açısından önemli olabileceğini gösterdi. Kullanıcının neye izin verdiğini açıklarken, görünmeyen bileşenlere ait asenkron işlemlerin de uygulamanın geri kalanını etkilememesine dikkat etmeyi öğrendim.

Çalışılan Tarih	Onaylayanın Adı Soyadı	İmza ve Mühür
[GG/AA/YYYY]	[SORUMLU AD SOYAD]	

Otomatik Testler ve Hata Düzeltmelerinin Doğrulanması

Bugün uygulamanın doğrulama çalışmalarını test türlerine göre değerlendirdim. Birim testleri miktar hesaplama, JSON dönüşümü ve hedef uyarılama gibi arayüzden bağımsız kuralları denetliyor. Widget testleri ise ekranı kontrollü ortamda açıp düğmeye basma, metin girme ve geri dönme gibi etkileşimleri sınavabiliyor. Aynı özelliği hem model hem ekran düzeyinde ele almak, hesabın doğru olmasına rağmen kullanıcı akışında oluşan hataları fark etmeyi sağlıyor.

Test verilerinde gerçek günlük veya canlı yapay zekâ yanıtı yerine sabit besin nesneleri, sahte servisler ve bellek içi depolama kullandım. Beklenen sonucu böylece önceden hesaplayabiliyorum. Örneğin kalori testinde 132 sonucunu, düzenleme testinde aynı kayıt kimliğini, tekrar istekte ise tek besin kalmasını arıyorum. Sadece ekranda hata görünmemesi değil, verinin beklenen sayıya ve duruma sahip olması kontrol ediliyor.

Asenkron sorunlar için Completer ile yanıtı ve depolamayı istediğim aşamada bekleten senaryoları inceledim. Arama temizlendikten sonra geç gelen sonuç, başarısız düzeltme sonrası geri dönüş ve araç takımı kapanmadan önce yerel kaydın tamamlanması bu yöntemle sınanıyor. Böylece doğal kullanımda zamanlamaya bağlı ortaya çıkan hatalar, aynı sırayla tekrar üretilebilir hâle geliyor. Düzeltmeden sonra aynı testi korumak, hatanın ileride geri dönmesini yakalamaya yardımcı oluyor.

Doğrulama kaydında 14 Eylül 2026 tarihinde flutter analyze --no-pub temiz sonuç verdi; flutter test --no-pub --reporter expanded ile 44 Flutter testi geçti. Bu sonuçları bütün telefonlarda veya her konuşmada kusursuz çalışma garantisi olarak yorumlamadım. Gerçek mikrofon gürültüsü, başlatıcı davranışı ve sağlayıcı sürekliliği ayrı denemeler gerektiriyor. Gün sonunda test yazmanın yalnızca son aşama olmadığını öğrendim: beklenen davranışı açıkça tanımlamak, uygulamadaki tasarım kararlarını da daha anlaşılır hâle getiriyor.

Çalışılan Tarih	Onaylayanın Adı Soyadı	İmza ve Mühür
[GG/AA/YYYY]	[SORUMLU AD SOYAD]	

Android Paketinin Hazırlanması ve Genel Değerlendirme

Bugün çalışmanın Android paketleme ve teslim tarafını ele aldım. pubspec.yaml içindeki 1.1.2+5 bilgisinde kullanıcıya gösterilen sürümle derleme numarasını ayırdım. flutter build apk --release komutu kurulum paketini üretir; uygulamaya ait herkese açık kimlik doğrulama yapılandırması derleme sırasında verilir. Gizli sunucu anahtarlarının pakete eklenmemesi gerekiyor. APK'nın üretilmesiyle mağazada yayımlanmasının farklı aşamalar olduğunu da not ettim.

Paket çıktısının build/app/outputs/flutter-apk/app-release.apk altında oluştuğunu ve güncellemede adb install -r komutunun mevcut uygulamanın üzerine kurulum için kullanıldığını inceledim. 5 Eylül denetim kaydı bu sürümün kurulum ve cihaz kontrollerini içeriyor; belge hazırlanırken yeni telefon kurulumu yapılmadı. Mevcut beta yerel debug imzasıyla hazırlanıyor. Mağaza dağıtımı için ayrı yayın anahtarı ve dağıtım hazırlığı gerektiğini ayırdım.

Teslim akışını uygulamanın günlük kullanım sırasına göre topladım: besin arama, porsiyon seçme, kaydı düzenleme, günlük toplamı görme, sesli kayıt ve araç takımıyla hızlı giriş. Ardından hedef ve diyet seçimiyle günlük öneri, koç görüşmesi ve Oracle bağlantısını ilişkilendirdim. README kullanım ve doğrulama komutlarını, davranış belgesi mimariyi, denetim kaydı test sonuçlarını, BACKLOG ise tamamlanmamış işleri açıklıyor. Böylece sonraki kişi yalnızca kurulum dosyasına bağımlı kalmıyor.

Genel değerlendirmede yerel yedekleme, yarım kalan ses işlerinin kalıcı kuyruğu, daha kapsamlı cihaz denemeleri ve kaynak verisiyle puanlanan Oracle gibi geliştirme alanlarını belirttim. Bunları bitmiş özellikler gibi sunmadım. Bu çalışma boyunca arayüz, API, cihaz özellikleri, hesaplama ve testlerin birbirine bağlı olduğunu öğrendim. En önemli kazanımım, bir özelliği yalnızca çalışır göstermekle yetinmeyip hangi veriyle, hangi koşulda ve hata olduğunda nasıl davrandığını açıklayabilecek bir geliştirme yaklaşımı edinmek oldu.

Çalışılan Tarih	Onaylayanın Adı Soyadı	İmza ve Mühür
[GG/AA/YYYY]	[SORUMLU AD SOYAD]	